Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Automation of processes

- DevOps methodology introduce some key concepts about the automatization of repetitive processes, in particular
  - Continuous Integration (CI)
  - Continuous Delivery (CD)
  - Continuous Deployment
- These automatizations in order to speedup the activities, increase the quality of the code and obtain fast feedback ("fail fast -> learn fast -> improve fast" – "make mistakes faster than anyone else")

Continuous Integration

- Continuous Integration is the practice of frequently merging all developers' working copies into a shared main branch. It involves automated testing to ensure that new code changes integrate smoothly and do not break the existing codebase. CI helps to detect and fix errors quickly, making the integration process smoother and less prone to bugs.

Continuous Integration

- Each integration triggers an automated build that verifies the new code against existing functionality, helping to identify errors as early as possible. The goal of CI is to ensure that the codebase remains stable and that any issues are caught quickly before they affect other team members.

Continuous Integration - tools

- Jenkins: An open-source automation server that is widely used for CI. It provides a robust plugin ecosystem and can be integrated with various tools for version control, testing, and deployment.
- GitLab CI: A CI/CD tool integrated with GitLab repositories. It allows teams to define CI/CD pipelines using a .gitlab-ci.yml file and offers an easy setup process for automating builds, testing, and deployments.
- CircleCI: A cloud-based CI/CD platform that integrates with GitHub, Bitbucket, and other version control systems. CircleCI is known for its speed and ease of use, offering fast build times and easy configuration with YAML files.

Continuous Integration - Demo

CI based on pipeline through GitHub Actions

Configuration file - yaml

```yaml
name: 20241012-devmet24-CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v2

    - name: Use Node.js
      uses: actions/setup-node@v2
      with:
        node-version: '16.x'

    - name: Install dependencies
      run: |
        npm ci
        npm install react-scripts

    - name: Build
      run: npm run build
```

- yaml: YAML (Yet Another Markup Language, later redefined as YAML Ain't Markup Language) is a human-readable data serialization format that is often used for configuration files.
- file saved in .github/workflows/
- name: name of workflow
- on: when the workflow has to be executed
- jobs: define the jobs to execute
- build: name of the job
- runs-on: execution environment
- steps: actions to execute during the job

Continuous Integration - Demo

CI based on pipeline through Jenkins

Ca' Foscari University of Venice

Jenkins installation

- Create an EC2 virtual machine

```
# Installing and Configuring Jenkins on Ubuntu

## 1. Update the system

```bash
sudo apt update
sudo apt upgrade
```

2. Install Java

Jenkins requires Java. Let's install OpenJDK:

```bash
sudo apt install openjdk-11-jdk
```

3. Add the Jenkins repository

```bash
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```
```

```

## 4. Install Jenkins

```bash
sudo apt update
sudo apt install jenkins
```

5. Start Jenkins

```bash
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

6. Open the firewall port

If you have an active firewall, open port 8080:

```bash
sudo ufw allow 8080
```

7. Access the web interface

Open a browser and go to `http://your_ip_address
```

Continuous Delivery

- Continuous Delivery is an extension of Continuous Integration. It ensures that the codebase is always in a deployable state by automatically pushing changes to a staging environment where they can be tested and validated. The goal of Continuous Delivery is to have a deployment pipeline that is ready to release new features or updates to production at any time with minimal manual intervention.

Continuous Deployment

- Continuous Deployment takes Continuous Delivery a step further by automatically deploying every change that passes the automated tests directly to production. This process ensures that new code reaches end-users as soon as it is ready, enabling faster feedback loops and rapid delivery of new features. Continuous Deployment requires a high level of automation and robust testing to maintain production stability.

Ca' Foscari University of Venice

Continuous Delivery vs Deployment

- **Level of Automation**
  - **Continuous Delivery:** In Continuous Delivery, every change that passes automated tests is automatically pushed to a staging environment or a pre-production environment. However, releasing to production still requires a manual approval step. This gives teams control over when and what gets released to users, allowing for scheduled releases.
  - **Continuous Deployment:** Continuous Deployment automates the entire process, including the release to production. Every change that passes the tests is automatically deployed to production without any manual intervention. This allows new features or bug fixes to reach users as soon as they are ready.
- **Risk and Control:**
  - **Continuous Delivery:** Offers more control over the release process, making it easier to manage and review changes before they reach end-users. This is particularly useful for teams that want to ensure higher stability or have a slower release cycle.
  - **Continuous Deployment:** Carries a higher level of risk because changes are released directly to users without a manual review step. However, it also allows for faster feedback loops and quick response to issues, as new updates can be delivered immediately.
- **Use Cases and Suitability:**
  - **Continuous Delivery:** Ideal for teams that want to automate the testing and integration processes but prefer to schedule releases according to business needs or customer readiness. It's suitable for environments where a more controlled release process is required.
  - **Continuous Deployment:** Works best in environments where speed is a priority, and the team is confident in their automated testing and monitoring processes. It is well-suited for applications where frequent updates and fast iterations are needed, such as web-based services or SaaS products.
- In summary, while both practices aim to streamline the delivery process, Continuous Delivery retains a manual gate before production, whereas Continuous Deployment removes this gate, making every change automatically available to users.

Continous Delivery pipeline

- The Continuous Delivery (CD) pipeline is closely connected to the Continuous Integration (CI) process, forming a streamlined workflow that allows code changes to move smoothly from development to production. Here's how the CD pipeline links to the CI process

CI Stage

- The CI process begins when developers push their code changes to a shared version control repository (e.g., Git).
- This triggers automated builds and tests through a CI pipeline. The CI pipeline typically includes steps like:
  - Build: Compiling the code or packaging it into deployable units, such as Docker images.
  - Automated Tests: Running unit tests, integration tests, and possibly code quality checks to ensure that the changes don't break existing functionality.
- If the build and tests pass successfully, the code is considered integrated and ready for deployment.

Transition to the CD pipeline

- Once the code is validated by the CI process, the CD pipeline takes over. It uses the artifacts generated by CI (e.g., Docker images, executables) and prepares them for deployment.
- The CD pipeline's role is to ensure that the validated code is deployed in a consistent and reliable manner to different environments, such as staging or production.

Stages of the CD pipeline

- Staging/Testing Environment Deployment: The code is deployed to a staging environment, where further automated or manual tests might be performed, such as integration tests, end-to-end tests, or user acceptance testing (UAT).
- Approval Step (Continuous Delivery): If the organization practices Continuous Delivery, a manual approval step may be required before moving the code to production. This is common when a business needs control over the release timing.
- Production Deployment (Continuous Deployment): In a Continuous Deployment setup, this approval step is automated, and successful changes are automatically deployed to production.

Automated Rollout and Monitoring

- After deployment, monitoring tools may be used to check the performance and stability of the newly deployed code in production. This step ensures that any issues are identified quickly and can trigger rollbacks if necessary.

Strategies for Deployment Automation

- Automating deployment is crucial for ensuring that software updates are delivered reliably, quickly, and safely
- Let's see common strategies for deployment automation.

Automated Testing in the Deployment Pipeline

- *Overview*: Integrating automated tests directly into the deployment pipeline ensures that new releases are validated before reaching users. These tests can include smoke tests, integration tests, and end to end tests.
- *Deployment Process*: As new code moves through the CI/CD pipeline, each stage includes specific tests that must pass before the code is deployed to the next environment (e.g., from staging to production).
- *Benefits*: Automated testing in the pipeline reduces the risk of deploying broken code to production. It ensures that each release meets quality standards before being deployed, providing an additional layer of safety.
- *Example*: After building a new version of a web application, the CI/CD pipeline could run a suite of end-to-end tests that simulate user interactions. If all tests pass, the deployment to the production environment proceeds automatically.

CI/CD Tools for Deployment Automation

- *Overview*: Tools like Jenkins, GitLab CI/CD, CircleCI, and GitHub Actions help automate the deployment process. They allow teams to define CI/CD pipelines in configuration files (e.g., YAML), specifying how code should be built, tested, and deployed.
- *Deployment Process*: These tools can trigger deployments based on events like code pushes, merges, or scheduled times. They manage the entire workflow, including build automation, test execution, artifact creation, and deployment to target environments.
- *Benefits*: CI/CD tools streamline the deployment process, allowing for greater efficiency and reducing manual intervention. They also provide insights and logs for each step of the pipeline, making it easier to diagnose and resolve issues.
- *Example*: Using GitLab CI, a .gitlab-ci.yml file can define a deployment pipeline that automatically deploys a Docker image to a Kubernetes cluster when changes are merged into the main branch.

Infrastructure as Code (IaC)

- *Overview*: Infrastructure as Code (IaC) allows teams to define and manage their infrastructure (servers, databases, networks, etc.) using code rather than manual processes. Tools like Terraform, Ansible, AWS CloudFormation, and Pulumi are commonly used for this purpose.
- *Benefits*: With IaC, infrastructure configurations are stored in version control systems (like Git), making them reproducible and trackable. This ensures that environments are consistent, whether they are for development, staging, or production, reducing the risk of environment-specific bugs.
- *Example*: Using Terraform, you can write a script that provisions a new server, configures network settings, and deploys an application automatically. When a new version of the application is ready, a simple command can update the infrastructure and deploy the latest code without manual intervention.

Blue-Green Deployments

- *Overview*: This strategy involves maintaining two identical environments: one active (blue) and one inactive (green). The green environment is updated with the new version of the software, tested, and verified while the blue environment continues serving users.
- *Deployment Process*: Once the green environment is ready, traffic is switched from the blue environment to the green environment, making the new version live. The blue environment remains on standby as a fallback option.
- *Benefits*: This method ensures minimal downtime since the switch between environments is nearly instantaneous. It also provides a quick rollback mechanism—if issues are detected in the green environment after it goes live, traffic can be switched back to the blue environment.
- *Example*: For a web application, the blue environment might run version 1.0 while the green environment is updated to version 1.1. After testing version 1.1 on green, traffic is redirected to green using a load balancer, making version 1.1 the active version.

Canary Releases

- *Overview*: A canary release gradually rolls out the new version to a small subset of users before expanding to the entire user base. It allows you to test the new version with real user traffic while minimizing the risk of widespread issues.
- *Deployment Process*: The new version is first deployed to a small percentage of servers or user groups. If no issues are detected, the deployment is gradually expanded until 100% of users are on the new version.
- *Benefits*: Canary releases help in identifying potential issues with a new release before they affect the entire user base. This makes it easier to roll back changes if a problem is detected early in the process.
- *Example*: For a mobile app, a canary release might initially be rolled out to 5% of users. If no errors or performance issues are reported after a certain period, the release is gradually increased to 25%, 50%, and eventually 100% of users.

Rolling Updates

- *Overview*: A rolling update gradually replaces instances of the old version of an application with new instances, one or a few at a time. This approach allows you to update a service without taking it offline.
- *Deployment Process*: In a rolling update, a certain number of instances (or pods, in the case of Kubernetes) are updated while others continue to serve traffic. As each instance is updated and verified, the next set is updated.
- *Benefits*: Rolling updates allow for minimal disruption to end-users, as only a portion of the service is updated at a time. This is particularly useful for systems that require high availability and cannot afford downtime.
- *Example*: For a microservices-based application running on Kubernetes, a rolling update would involve updating a few pods to a new version. If the updated pods function correctly, more pods are updated until the entire deployment is complete.

Summary on strategies

- These strategies for deployment automation help teams ensure that their deployment process is fast, reliable, and scalable.
- Each strategy has its own strengths and is chosen based on the specific needs of the application and the level of risk tolerance.
- By automating deployments, organizations can achieve continuous delivery or continuous deployment, enabling rapid and safe delivery of software updates to users.

Tools for Deployment

- Automating deployments often involves a combination of tools that work together to package, orchestrate, and provision infrastructure.
- If we talking about automating deployments we have to talk about some tools like Docker, Kubernetes and Terraform

Docker: Containerization

- Overview: Docker is a platform that enables developers to package applications and their dependencies into containers. Containers are lightweight, isolated environments that run consistently across different platforms, making it easier to deploy applications without worrying about compatibility issues.
- Use Case: A developer can create a Docker image of a web application, which includes all necessary dependencies (like libraries, runtime, etc.). This image can then be deployed in any environment that supports Docker, ensuring that the application behaves the same on a developer's machine, a test server, or in production.

Docker

```
FROM node:14
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

- Running the Container: After building the Docker image with
- `docker build -t myapp .`
- the container can be run with
- `docker run -p 3000:3000 myapp`
- making the application accessible on port 3000.

- Benefits: Docker ensures consistency across development, testing, and production environments, making it easier to debug issues and manage application dependencies. Containers are also lightweight and can be easily scaled.

Kubernetes: Orchestration

- Overview: Kubernetes (often abbreviated as K8s) is an orchestration platform for managing containerized applications at scale. It allows you to deploy, manage, and scale applications by automating the deployment of Docker containers across clusters of servers.
- Use Case: In a production environment, a single Docker container might not be sufficient. Kubernetes can manage multiple instances of an application (pods), handle load balancing, perform rolling updates, and automatically recover from failures.

Kubernetes

- Benefits: Kubernetes handles scaling (both up and down), automatic failover, load balancing, and self-healing, making it ideal for production environments where availability and reliability are critical.

Ca' Foscari University of Venice

Kubernetes demo

- Install prerequisites
  - docker
  - kubectl (tool to manage k8s)
  - minikube (run locally k8s)
- `minikube start`
  - Start the service
- `docker build -t myapp:tag .`
  - Build docker
- `eval $(minikube docker-env)`
  - Configure Docker to use Minikube's Docker daemon
- `docker build -t myapp:tag .`
  - Rebuild your image in Minikube's context
- `kubectl apply -f deployment.yaml`
  - Apply the deployment
- Check the k8s-deploy.sh to see other commands.

Terraform: Infrastructure as Code (IaC)

- Overview: Terraform is an open-source tool that allows you to define, provision, and manage infrastructure as code. It enables you to describe your infrastructure in configuration files (written in HashiCorp Configuration Language, HCL), which can then be used to create and manage resources on cloud platforms like AWS, Azure, or Google Cloud.
- Use Case: Terraform is useful for setting up cloud resources like virtual machines, networks, databases, and load balancers in a reproducible and consistent manner. It helps automate the provisioning of these resources, which is particularly beneficial when deploying complex applications with multiple dependencies.

Terraform

- Benefits: Terraform enables teams to automate the provisioning of infrastructure, making it easier to maintain and reproduce environments. Changes to infrastructure can be tracked through version control, allowing for better collaboration and management of resources.

Terraform example

```
# Terraform file example
# Configure the AWS Provider
provider "aws" {
  region = "us-west-2"
}

# Create a VPC
resource "aws_vpc" "example_vpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "example-vpc"
  }
}

# Create a subnet
resource "aws_subnet" "example_subnet" {
  vpc_id     = aws_vpc.example_vpc.id
  cidr_block = "10.0.1.0/24"

  tags = {
    Name = "example-subnet"
  }
}
```

- Configures the AWS provider for the us-west-2 region.
- Creates a VPC with a CIDR block of 10.0.0.0/16.
- Creates a subnet within the VPC with a CIDR block of 10.0.1.0/24.

VPC = Virtual Private Cloud, a logically isolated section of the cloud where you can launch resources in a virtual network that you define. Your resources are separated from other users' resources in the cloud.

Ca' Foscari University of Venice

Terraform example

```
# Create a security group
resource "aws_security_group" "example_sg" {
  name        = "example-sg"
  description = "Allow inbound SSH traffic"
  vpc_id      = aws_vpc.example_vpc.id

  ingress {
    description = "SSH from VPC"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "allow_ssh"
  }
}

# Create an EC2 instance
resource "aws_instance" "example_instance" {
  ami           = "ami-0c55b159cbfafe1f0"  # Amazon Linux 2 AMI (HVM), SSD Volume Type
  instance_type = "t2.micro"
  subnet_id     = aws_subnet.example_subnet.id

  vpc_security_group_ids = [aws_security_group.example_sg.id]

  tags = {
    Name = "example-instance"
  }
}
```

- Sets up a security group that allows inbound SSH traffic.
- Launches an EC2 instance using the Amazon Linux 2 AMI, with a t2.micro instance type, in the created subnet and security group.

This is a basic example and doesn't include all best practices or necessary components for a production environment.

You might want to add more security measures, configure proper networking, and add other resources as needed for your specific use case.

Ca' Foscari University of Venice

Let's see a demo

- main.tf

```
# Configure the AWS Provider
provider "aws" {
  region = "us-west-2"
}

# Create an S3 bucket
resource "aws_s3_bucket" "example_bucket" {
  bucket = "my-terraform-example-bucket-${random_id.bucket_suffix.hex}"
}

# Generate a random suffix for the bucket name
resource "random_id" "bucket_suffix" {
  byte_length = 8
}

# Output the name of the created bucket
output "bucket_name" {
  value = aws_s3_bucket.example_bucket.id
}
```

- `terraform init` - This will download the necessary provider plugins.
- `terraform plan` - Plan your infrastructure
- `terraform apply` - To create the S3 bucket
- `terraform destroy` - To destroy the resources when you're done

How they work together

- These tools can be used in combination to automate the entire deployment process:
  - Docker packages the application into a portable container.
  - Kubernetes orchestrates these Docker containers, managing their deployment, scaling, and availability across clusters.
  - Terraform provisions the underlying infrastructure, such as virtual machines, networking, and storage, to run Kubernetes and other application components.

How they work together

- For example, you could use Terraform to set up an AWS EKS (Elastic Kubernetes Service) cluster, deploy your Docker-based microservices to the cluster using Kubernetes, and manage updates and scaling using Kubernetes' orchestration capabilities. This combined approach ensures that both the infrastructure and the application deployment are automated and manageable through code.

Ca' Foscari University of Venice

Rollback

- Implementing a controlled rollback is an important part of a robust deployment strategy
- Key points:
  - Versioning strategy
  - Artifact storage
  - Creating a rollback workflow
  - Manual trigger for rollback

```yaml
name: Rollback

on:
  workflow_dispatch:
    inputs:
      version:
        description: 'Version to rollback to'
        required: true

jobs:
  rollback:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v2

    - name: Download artifact
      uses: actions/download-artifact@v2
      with:
        name: app-build-${{ github.event.inputs.version }}
        path: ./build

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v1
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: us-east-1  # Sostituisci con la tua regione

    - name: Deploy to S3
      run: |
        aws s3 sync ./build s3://${{ secrets.AWS_S3_BUCKET }} --delete

    - name: Invalidate CloudFront
      run: |
        aws cloudfront create-invalidation --distribution-id ${{ secrets.CLOUDFRONT_DISTRIBUTION_ID }} --paths "/*"

    - name: Notify rollback
      uses: 8398a7/action-slack@v3
      with:
        status: ${{ job.status }}
        text: Rollback to version ${{ github.event.inputs.version }} completed
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK }}
```

Ca' Foscari University of Venice

Rollback

- This workflow:
- It is triggered manually (workflow_dispatch)
- It allows specifying the version to roll back to
- Downloads the artifact of the specified version
- Deploys it to S3
- Invalidates the CloudFront cache
- Notifies the completion of the rollback on Slack

```yaml
name: Rollback

on:
  workflow_dispatch:
    inputs:
      version:
        description: 'Version to rollback to'
        required: true

jobs:
  rollback:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v2

    - name: Download artifact
      uses: actions/download-artifact@v2
      with:
        name: app-build-${{ github.event.inputs.version }}
        path: ./build

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v1
      with:
        aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
        aws-region: us-east-1  # Sostituisci con la tua regione

    - name: Deploy to S3
      run: |
        aws s3 sync ./build s3://${{ secrets.AWS_S3_BUCKET }} --delete

    - name: Invalidate CloudFront
      run: |
        aws cloudfront create-invalidation --distribution-id ${{ secrets.CLOUDFRONT_DISTRIBUTION_ID }} --paths "/*"

    - name: Notify rollback
      uses: 8398a7/action-slack@v3
      with:
        status: ${{ job.status }}
        text: Rollback to version ${{ github.event.inputs.version }} completed
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK }}
```

Rollback

- To make this system even more robust, consider implementing:
  - Post-rollback smoke tests: Add a step in the rollback workflow that runs basic tests to ensure that the previous version functions correctly.
  - Database management: If your application uses a database, ensure that the rollback of the application is compatible with the current database schema. You might need to implement reversible migrations.
  - Feature flags: Use feature flags to enable/disable features without having to deploy a new version. This can reduce the need for full rollbacks.
  - Canary releases: Implement a canary release system, where the new version is initially distributed to only a small percentage of users. This can help identify issues before they impact all users.

Rollback

```yaml
- name: Generate build ID
  id: build_id
  run: echo "::set-output name=id::$(date +'%Y%m%d%H%M%S')"

- name: Archive build
  uses: actions/upload-artifact@v2
  with:
    name: app-build-${{ steps.build_id.outputs.id }}
    path: build
```

```yaml
- name: Create deployment tag
  run: |
    git config user.name github-actions
    git config user.email github-actions@github.com
    git tag -a v${{ steps.build_id.outputs.id }} -m "Deployment ${{ steps.build_id.outputs.id }}"
    git push origin v${{ steps.build_id.outputs.id }}
```

- This workflow:
  - Generates a unique build ID
  - Archives the build artifact with the unique ID
  - Creates a git tag for each deployment
- These changes allow for a clear history of deployments and make the rollback process easier.